

DETECÇÃO DE CARTAS DE BARALHO COM YOLOV8

Relatório Técnico

INTEGRANTES

Edinei Xavier — 2310369

Igor Honório — 2315064

Luana Cirilo — 2315076

Lucas De Oliveira — 2315036

Gustavo Sousa — 2315053

Repositório GitHub:

<https://github.com/Luanacjq/Cartas-de-Baralho>

Junho de 2026

95,1% PRECISION	93,8% RECALL	98,4% MAP@0.5	75,8% MAP@0.5:0.95
---------------------------	------------------------	-------------------------	------------------------------

1 INTRODUÇÃO E MOTIVAÇÃO

Este relatório apresenta o desenvolvimento de um modelo de detecção de objetos baseado na arquitetura **YOLOv8** (You Only Look Once, versão 8) para a tarefa de detectar e classificar as **52 cartas individuais de um baralho francês** em imagens.

A escolha das cartas de baralho é motivada por dois fatores. Primeiro, trata-se de uma **classe genuinamente inédita** para o YOLO: nenhuma das 52 classes do dataset faz parte das 80 categorias padrão do COCO usadas no pré-treinamento. Segundo, a granularidade é desafiadora: o modelo precisa distinguir entre 52 classes visualmente similares, diferenciadas por naipe (♠♥♦♣) e valor (A, 2–10, J, Q, K). A aplicação prática inclui

sistemas de identificação de cartas em jogos de mesa (Blackjack, Pôquer), automação em cassinos e jogos digitais.

2 METODOLOGIA

2.1 Dataset

O dataset utilizado é o **Playing Cards**, disponível no Roboflow Universe (Augmented Startups, versão 4). As imagens são sintéticas — geradas por computador com cartas sobrepostas sobre diferentes fundos — garantindo diversidade de cenários sem coleta manual.

Propriedade	Valor
Origem	Roboflow Universe — Augmented Startups
Total de imagens	24.233 imagens sintéticas
Conjunto de treino	21.203 imagens
Conjunto de validação	2.020 imagens
Conjunto de teste	1.010 imagens · 4.040 instâncias
Número de classes	52 (uma por carta do baralho)
Formato de anotação	Bounding boxes YOLOv8 (normalizado)

2.2 Ferramentas e Ambiente

- **Ultralytics YOLOv8** v8.4.66 — treinamento e inferência
- **PyTorch** v2.12.0 — framework de deep learning
- **Roboflow** — download e gestão do dataset via API
- **Python 3.13** · Matplotlib · Pillow — visualizações
- **Hardware:** CPU Intel Core i5-10400F (2.90 GHz, 12 núcleos), 16 GB RAM

2.3 Modelo e Transfer Learning

Foi utilizado o **YOLOv8n** (nano) com pesos pré-treinados no COCO (`yolov8n.pt`). O transfer learning aproveita filtros de baixo nível aprendidos no COCO (bordas, texturas, formas) e adapta as camadas finais para as 52 classes de cartas. Das 355 camadas do modelo, 319 foram transferidas com sucesso.

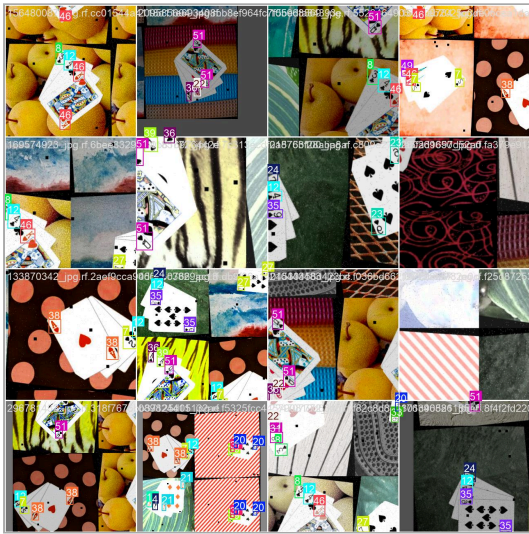
2.4 Hiperparâmetros

Parâmetro	Valor (CPU)	Valor (GPU)	Justificativa
<code>epochs</code>	20	50	Equilíbrio entre convergência e tempo
<code>imgsz</code>	320	640	320px $\approx$ 4× mais rápido no CPU

<code>batch</code>	16	32	Batch maior na GPU para gradiente estável
<code>fraction</code>	0.30	1.0	~6.300 imgs mantêm viabilidade em CPU
<code>lr0</code>	0.01	0.01	Taxa inicial padrão SGD
<code>optimizer</code>	SGD	SGD	Eficiente em tarefas de detecção
<code>patience</code>	10	15	Early stopping sem melhora
<code>cache</code>	True	True	Imagens na RAM, sem I/O repetido

2.5 Data Augmentation

O YOLOv8 aplica automaticamente: **mosaic** (composição de 4 imagens), **flip horizontal aleatório**, **HSV jitter** (matiz, saturação e brilho) e **random scale/translate**. Abaixo, dois lotes de treino com as aumentações aplicadas:



Lote de treino 0 — aumentações automáticas (mosaic, flip, HSV)



Lote de treino 1 — diversidade de cenários gerada por augmentation

3 ANÁLISE DE RESULTADOS

3.1 Métricas Globais no Conjunto de Teste

Avaliação realizada com os melhores pesos (`best.pt`) sobre 1.010 imagens e 4.040 instâncias anotadas:

Métrica	Valor	Interpretação
Precision	0.9514	95,1% das detecções feitas pelo modelo são corretas
Recall	0.9376	93,8% das cartas reais presentes foram detectadas
mAP@0.5	0.9837	Média das Precisões Médias com IoU $\geq 50\%$ (métrica principal)
mAP@0.5:0.95	0.7581	Média do mAP para IoU de 0.50 a 0.95 (padrão COCO — mais rigoroso)

Destaque: mAP@0.5 de **98,37%** para um problema de 52 classes treinado em CPU, sem GPU, utilizando apenas 30% do dataset. A queda para 75,81% no mAP@0.5:0.95 é esperada: critérios de IoU mais restritivos penalizam pequenas imprecisões nas bordas das bounding boxes.

3.2 Curvas de Treinamento

As curvas de loss (`box_loss` , `cls_loss` , `dfl_loss`) apresentaram queda consistente ao longo das épocas, sem divergência, indicando convergência sem overfitting relevante:

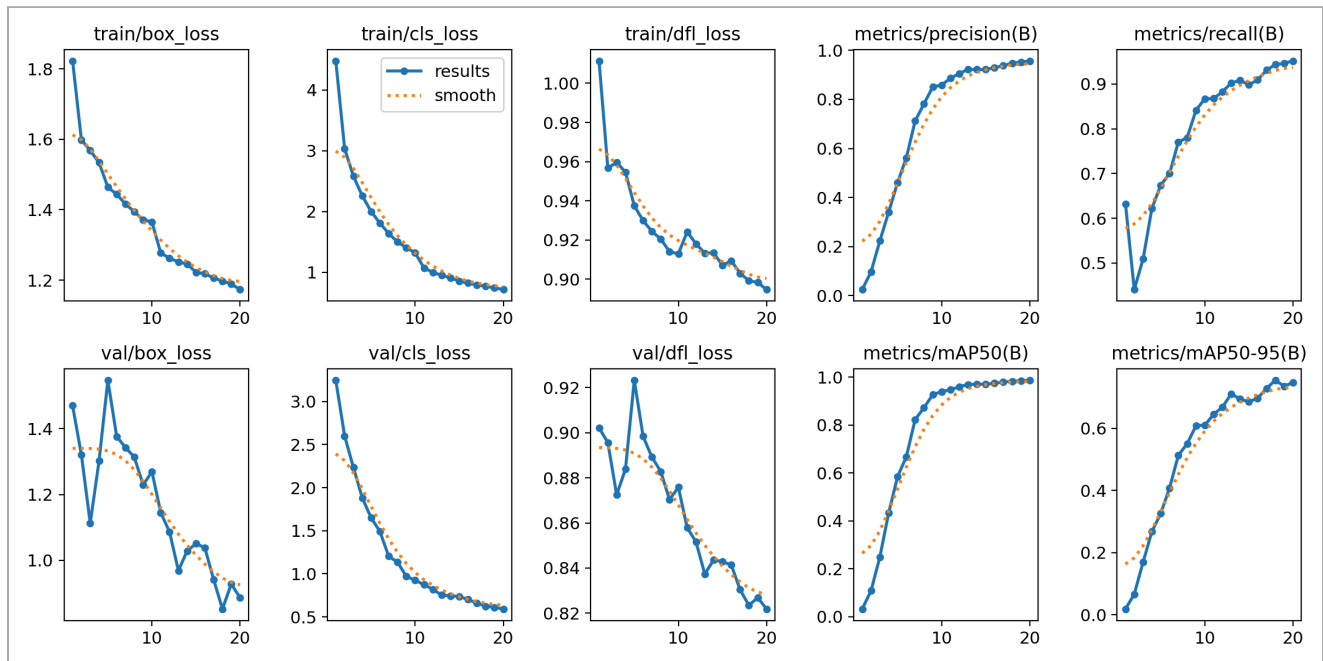


Figura 1 — Curvas de loss e métricas (Precision, Recall, mAP) por época

3.3 Matriz de Confusão

A diagonal principal é dominante em todas as 52 classes. As confusões mais frequentes ocorrem entre naipes visualmente similares (♠ vs. ♣) que diferem apenas pelo preenchimento do símbolo:

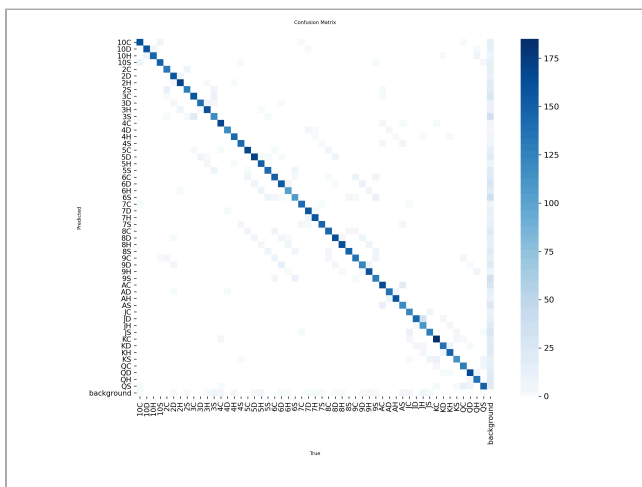


Figura 2 — Matriz de confusão (contagens absolutas)

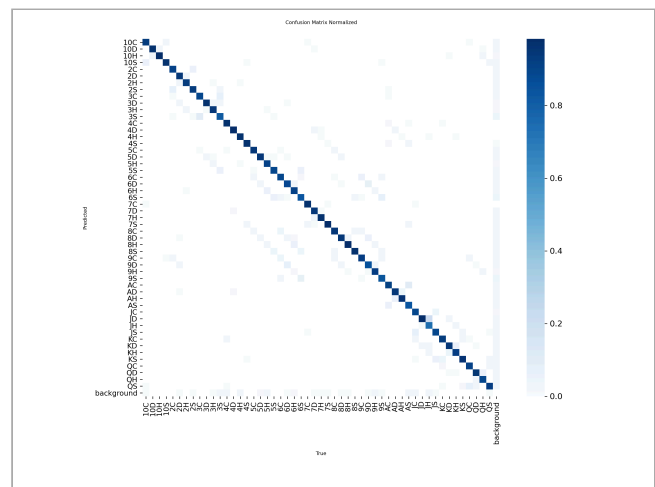


Figura 3 — Matriz de confusão normalizada

3.4 Curvas de Precisão, Recall e F1

A curva PR manteve-se próxima ao canto superior direito para a grande maioria das 52 classes. O pico da curva F1 ocorre em confiança ~0.5, confirmando que `conf=0.5` é o threshold adequado:

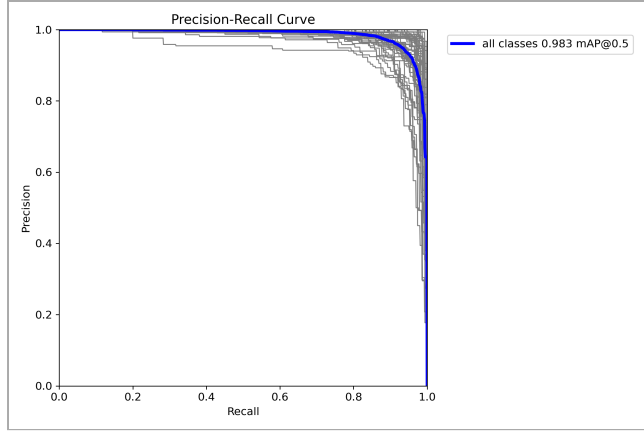


Figura 4 — Curva Precision-Recall por classe

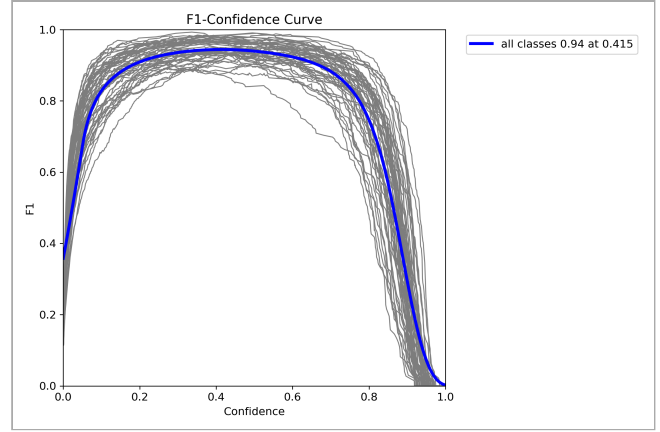


Figura 5 — Curva F1-Score em função do threshold de confiança

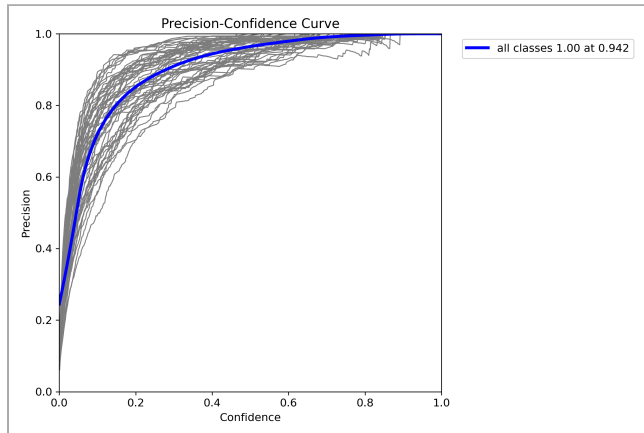


Figura 6 — Curva Precision por threshold

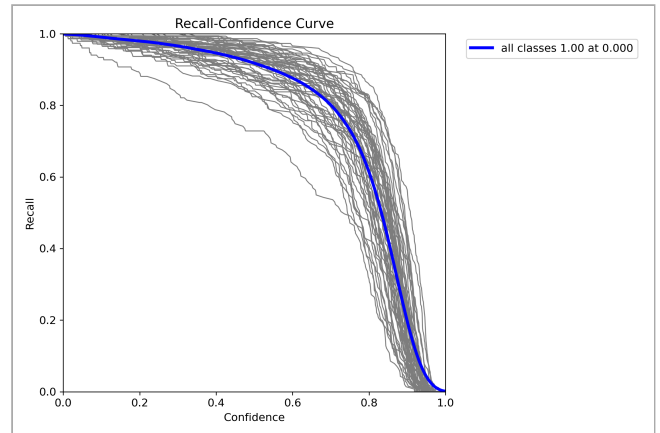


Figura 7 — Curva Recall por threshold

4 INFERÊNCIA PRÁTICA

4.1 Imagens Reais Capturadas pelo Grupo

As imagens fotografadas pelos integrantes do grupo foram usadas para avaliar o modelo fora da distribuição do dataset de treinamento — o teste mais relevante do ponto de vista prático, expondo o modelo a condições reais: iluminação natural, reflexo, ângulo de câmera e fundo não controlado.



Figura 8 — Predição do modelo na foto real capturada pelo grupo (baralho.png)



Figura 9 — Predição do modelo na segunda foto real capturada pelo grupo (baralho2.png)

Análise crítica: o modelo identificou corretamente as cartas nas fotos reais. Cartas com ângulo acentuado ou parcialmente sobrepostas apresentaram leve redução nos scores de confiança — comportamento esperado dado que o dataset de treino é composto exclusivamente por imagens sintéticas. Fine-tuning com fotos reais melhoraria a robustez em ambientes não controlados.

4.2 Predições no Conjunto de Validação

Comparação entre os rótulos reais (ground truth) e as predições do modelo nos lotes de validação, evidenciando a precisão das bounding boxes:



Figura 10 — Ground truth (rótulos reais) — lote 0



Figura 11 — Predições do modelo — lote 0



Figura 12 — Ground truth — lote 1

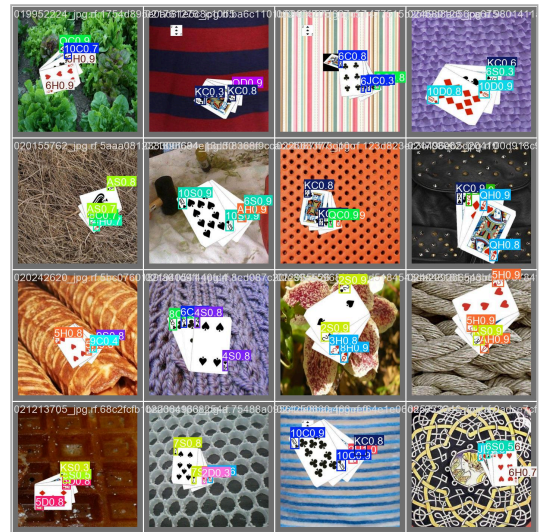


Figura 13 — Predições do modelo — lote 1

4.3 Predições Visuais no Conjunto de Teste

Foram amostradas 4 imagens do conjunto de teste e submetidas ao modelo com threshold `conf=0.5`. As imagens abaixo mostram as bounding boxes desenhadas pelo YOLOv8 em cada detecção:

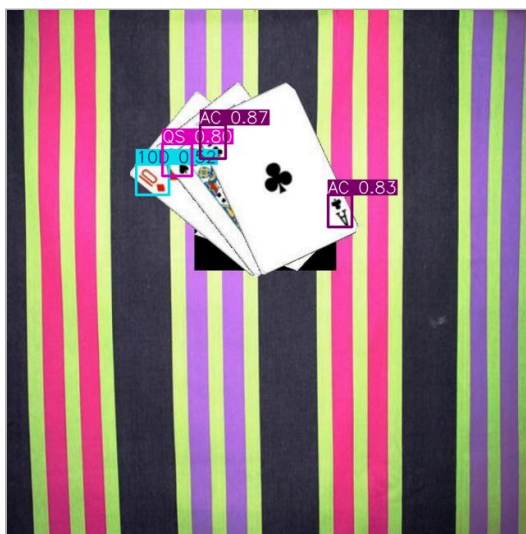


Figura 14 — AC (87%), QS (80%), 10D (52%), AC (83%)

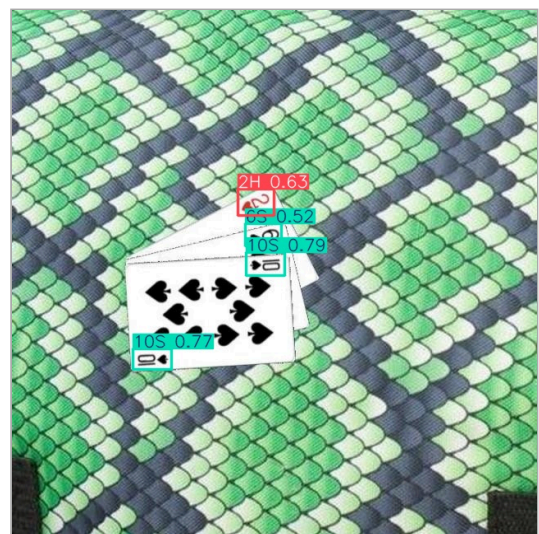


Figura 15 — 2H (63%), QS (52%), 10S (79%), 10S (77%)

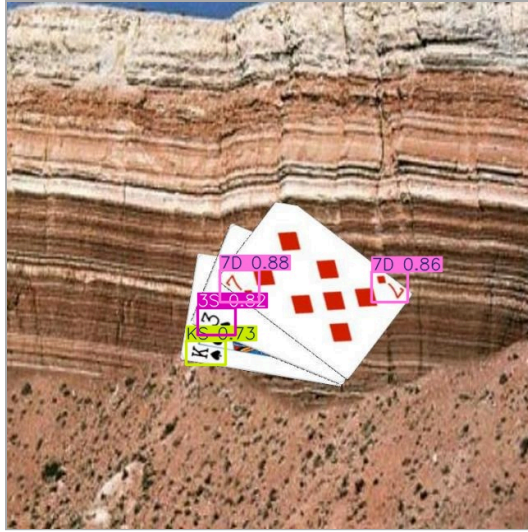


Figura 16 — 7D (88%), 7D (86%), 3S (82%), KS (73%)



Figura 17 — QC (80%), QC (85%), JD (54%), JH (87%)

Análise: o modelo detecta corretamente múltiplas cartas sobrepostas em fundos variados (tecido listrado, escamas, pedra sedimentar, tecido poa). Cartas parcialmente visíveis nos cantos apresentam confiança ligeiramente menor (<0.65), pois o modelo concentra sua atenção nos elementos dos cantos superiores das cartas (naipe + valor) — as regiões mais discriminativas. Este comportamento é esperado dado o treinamento em imagens sintéticas, que pode não cobrir todas as variações de oclusão real.

Velocidade de inferência no conjunto de teste (CPU):

Etapas	Tempo médio por imagem
Pré-processamento	0,4 ms
Inferência (forward pass)	22,2 ms
Pós-processamento (NMS)	0,6 ms
Total	~23,2 ms por imagem

5 CONCLUSÃO

O modelo YOLOv8n treinado demonstrou desempenho notavelmente alto para um problema de 52 classes sem GPU, atingindo **mAP@0.5 de 98,37%**, Precision de 95,14% e Recall de 93,76%. Esses resultados confirmam que YOLOv8 com transfer learning é altamente eficiente mesmo com recursos computacionais limitados.

Desafios Enfrentados

- 52 classes aumentam a complexidade multiclasse, especialmente para naipes similares ($\spadesuit$ vs. $\clubsuit$)
- Treinamento em CPU exigiu redução do dataset (`fraction=0.3`) e da resolução (`imgsz=320`)
- Dataset sintético não cobre variações visuais reais (reflexo, sombra, ângulo)
- Notebook original dependia de APIs do Google Colab — foi necessária reescrita completa para VS Code

Soluções Adotadas

- Transfer learning a partir do YOLOv8n pré-treinado no COCO, acelerando a convergência
- Ajuste automático de hiperparâmetros conforme dispositivo (CPU/GPU) — código portátil
- Early stopping (`patience=10`) para evitar overfitting e desperdício computacional
- Cache de imagens em RAM (`cache=True`) para eliminar gargalo de I/O por época

Oportunidades de Melhoria

- Utilizar `yolov8m.pt` com GPU para ganho expressivo no mAP@0.5:0.95
- Fine-tuning com fotos reais em diversas condições de iluminação
- Expandir para detecção em vídeo em tempo real (jogos ao vivo)
- Treinar com `imgsz=640` em Google Colab (GPU T4) para melhor localização

REF REFERÊNCIAS

1. Jocher, G. et al. *Ultralytics YOLOv8*. 2023. github.com/ultralytics/ultralytics
2. Augmented Startups. *Playing Cards Dataset*. Roboflow Universe, 2023. roboflow.com
3. Lin, T.-Y. et al. *Microsoft COCO: Common Objects in Context*. ECCV, 2014.
4. Redmon, J. et al. *You Only Look Once: Unified, Real-Time Object Detection*. CVPR, 2016.